

5G NR Radar

Steve Blandino



WIRELESS NETWORKS DIVISION, COMMUNICATION TECHNOLOGY LAB
NATIONAL INSTITUTE OF STANDARDS AND TECHNOLOGY,
GAITHERSBURG, MD, USA

JUNE, 2025

Licensing Statement

NIST-developed software is provided by NIST as a public service. You may use, copy, and distribute copies of the software in any medium, provided that you keep intact this entire notice. You may improve, modify, and create derivative works of the software or any portion of the software, and you may copy and distribute such modifications or works. Modified works should carry a notice stating that you changed the software and should note the date and nature of any such change. Please explicitly acknowledge the National Institute of Standards and Technology as the source of the software.

NIST-developed software is expressly provided "AS IS." NIST MAKES NO WARRANTY OF ANY KIND, EXPRESS, IMPLIED, IN FACT, OR ARISING BY OPERATION OF LAW, INCLUDING, WITHOUT LIMITATION, THE IMPLIED WARRANTY OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, AND DATA ACCURACY. NIST NEITHER REPRESENTS NOR WARRANTS THAT THE OPERATION OF THE SOFTWARE WILL BE UNINTERRUPTED OR ERROR-FREE, OR THAT ANY DEFECTS WILL BE CORRECTED. NIST DOES NOT WARRANT OR MAKE ANY REPRESENTATIONS REGARDING THE USE OF THE SOFTWARE OR THE RESULTS THEREOF, INCLUDING BUT NOT LIMITED TO THE CORRECTNESS, ACCURACY, RELIABILITY, OR USEFULNESS OF THE SOFTWARE.

You are solely responsible for determining the appropriateness of using and distributing the software and you assume all risks associated with its use, including but not limited to the risks and costs of program errors, compliance with applicable laws, damage to or loss of data, programs or equipment, and the unavailability or interruption of operation. This software is not intended to be used in any situation where a failure could cause risk of injury or damage to property. The software developed by NIST employees is not subject to copyright protection within the United States.

5G NR Radar Documentation

5G New Radio Radar (5G NRad) is a MATLAB-based simulation framework that models the radar sensing capabilities of a 5G New Radio (NR) base station, as presented in:

“Detecting Airborne Objects with 5G NR Radars”, S. Blandino et al., (submitted to) IEEE MILCOM 2025.

5G NRad implements a monostatic sensing configuration in which a single 5G NR base station (BS) functions as both transmitter and receiver to detect and localize airborne Sensing Targets (STs) within its coverage area. The BS transmits standardized 5G NR communication signals and processes the received echoes scattered by targets in the scene. Target position is estimated by extracting the angle of arrival (AoA) and round-trip delay from the received signals.

To generate sensing observations, 5G NRad uses the Positioning Reference Signal (PRS) defined in the 5G NR standard TS 38.211. The platform supports realistic deployment scenarios, including Urban Micro (UMi) and Urban Macro (UMa), and integrates 3GPP-compliant models for path loss, shadow fading, and line-of-sight (LoS) probability based on TR 38.901 and TR 36.777. In addition, 5G NRad incorporates recent 3GPP developments related to Release 19, including updated radar cross-section (RCS) models for airborne targets and monostatic background clutter synthesis.

Key Features:

- End-to-end radar processing chain: PRS generation, Orthogonal Frequency-Division Multiplexing (OFDM) transmission, analog beamforming, clutter suppression, and target detection.
- 3GPP-based environmental modeling: supports Line-of-Sight (LoS) / Non-Line-of-Sight (NLoS) propagation, shadowing, fading, and multipath.
- Range-Doppler processing based detection.
- Evaluation support for both UMi and UMa deployment scenarios with configurable Unmanned Aerial Vehicle (UAV) altitudes.

1 Requirements

1.1 MATLAB version

5GNRad is developed in MATLAB and validated on **MATLAB R2024b**. Earlier releases may work, but are not officially supported.

1.2 Required toolboxes

The following MathWorks products are required for full functionality:

- **MATLAB**
- **5G Toolbox** (NR numerology, PRS generation, OFDM grid mapping)
- **Phased Array System Toolbox** (array objects, steering vectors, angle processing utilities)
- **Signal Processing Toolbox** (windowing and FFT-related helpers)

1.3 Optional toolboxes

- **Parallel Computing Toolbox** (parallel execution across drops via `parfor`)

2 Folder Structure

```

<repo-root>/
  main.m
  setup.m
  src/
    +nrRadar/
      +cfg/          (scenario configuration readers/validators)
      +rx/           (RX synthesis, OFDM demod, RD/RDA formation)
      +sens/         (CFAR, NMS, DOA, clustering, sidelobe suppression)
      +array/        (array utilities: steering, coordinate conventions)
      +io/           (export utilities)
      +internal/     (performance-oriented precompute and per-drop workers)
  examples/
    <scenarioName>/
      Input/
        simulationConfig.txt
        prsConfig.txt
        sensingConfig.txt
        txAntennaModel.json
        rxAntennaModel.json
        backgroundChannel.json  (optional)
        targetChannel.json      (optional)
        bsConfiguration.txt     (if applicable)
        targetConfiguration.txt (if applicable)
      Output/                   (generated results)

```

The 5G NRad repository is organized to support reproducible, scenario-driven simulations. The top-level layout is:

- **main.m**: Main script and entry point to run a batch of example scenarios.
- **setup.m**: Convenience script that adds **src/** to the MATLAB path.
- **src/**: MATLAB source code (package **+nrRadar**) implementing configuration parsing, waveform generation, channel application, receive processing, detection, and evaluation.
- **examples/**: Ready-to-run scenario folders. Each scenario contains an **Input/** folder with configuration files and (optionally) channel JSON files.

The implementation follows a “config → precompute → per-drop processing → export” workflow:

1. **Configuration:** read `Input/` files and validate parameters.
2. **Precompute:** build static objects reused across drops (e.g., PRS waveform, OFDM numerology, FFT grids, constants).
3. **Per-drop processing:** apply channel, form range–Doppler–angle products, and run detection/estimation.
4. **Export:** write outputs (detections and metrics) to `Output/`.

The MATLAB source code implementing the 5G NR PRS radar simulation workflow is organized as a MATLAB package under:

- `src/+nrRadar/+cfg/`: Scenario configuration readers and validators (simulation/PRS/sensing configuration, geometry).
- `src/+nrRadar/+rx/`: Receive processing (waveform synthesis, channel convolution, OFDM demodulation, RD/RDA formation).
- `src/+nrRadar/+sens/`: Sensing algorithms (CFAR, peak picking/NMS, DOA estimation, clustering, sidelobe suppression, geometry reconstruction).
- `src/+nrRadar/+array/`: Array utilities (steering vectors, coordinate conventions, alias handling).
- `src/+nrRadar/+io/`: Import/export utilities (scenario I/O, results export).
- `src/+nrRadar/+internal/`: Performance-oriented internal entry points (precompute and per-drop workers, `parfor`-safe).

In `examples/`, predefined example scenarios illustrate how to configure and run the software. Each scenario is self-contained and includes an `Input/` folder (and an `Output/` folder generated after running):

- `examples/<scenarioName>/Input/`: Scenario configuration and optional channel descriptions, typically including:
 - `simulationConfig.txt`: System and runtime settings (carrier, bandwidth, noise figure, etc.).
 - `prsConfig.txt`: PRS resource configuration (periodicity, comb size, symbol mapping, etc.).
 - `sensingConfig.txt`: Radar processing and detection parameters (FFT sizes, CFAR, DOA method, thresholds, etc.).
 - `txAntennaModel.json`, `rxAntennaModel.json`: Antenna/array and beamforming model definitions.

- `backgroundChannel.json` (optional): Environment/background MPCs (when using the uploaded-channel mode).
- `targetChannel.json` (optional): Target MPCs (when using the uploaded-channel mode).
- `bsConfiguration.txt`: Base station/TRP configuration (position/orientation).
- `targetConfiguration.txt` : Target truth states (position/velocity timeline).
- `examples/<scenarioName>/Output/`: Generated results produced by `nrRadar.io.exportResults(...)` (detections, statistics, and optional intermediate products).

Note: Channel JSON files are placed directly in each scenario's `Input/` folder.

3 Configuration files and parameters

Each scenario provides configuration files under `examples/<scenarioName>/Input/`. Text configuration files follow a two-column format (`parameter<TAB>value`).

3.1 `simulationConfig.txt`

The simulation configuration controls carrier, bandwidth, noise figure, runtime sizing, and scenario options.

`fc` Carrier frequency in Hz.

`systemBw` RF bandwidth in Hz.

Noise Figure (`systemNF`) Receiver noise figure in dB. Allowed range: 0–20 dB; default: 7 dB.

`channelScenario` Scenario identifier (e.g., `UMiAV`, `UMaAV`, `RMaAV`) used only if target MPCs are not provided.

`nStDrop` Number of state samples per drop (used to group snapshots into processing blocks).

`nMaxDrop` Max number of drops. The max number of drops is determined by the minimum between `nMaxDrop` and the number of drops possible given the number of targets and `nStDrop`.

3.2 `prsConfig.txt`

Defines the Positioning Reference Signal (PRS) parameters:

- **PRSResourceSetPeriod**: Slot periodicity and offset, specified as `[TPRSPeriod, TPRSOffset]`. `TPRSPeriod` must be one of `{4, 5, 8, 10, 16, 20, 32, 40, 64, 80, 160, 320, 640, 1280, 2560, 5120, 10240}`, and `TPRSOffset` must be in the range `[0, TPRSPeriod - 1]`. Default: `[1, 0]`.
- **PRSResourceOffset**: Slot offset of each PRS resource relative to the PRS resource set offset. Range: 0–511; default: 0.
- **PRSResourceRepetition**: Number of times each PRS resource is repeated. Allowed values: `{1, 2, 4, 6, 8, 16, 32}`; default: 1.
- **PRSResourceTimeGap**: Slot offset between consecutive repeated instances of a PRS resource. Allowed values: `{1, 2, 4, 8, 16, 32}`; default: 1.

- **NumRB**: Number of Physical Resource Blocks (PRBs) allocated for PRS. Range: 0–275; default: 66.
- **RBOffset**: Starting PRB index for PRS allocation. Range: 0–274; default: 0.
- **CombSize**: PRS comb size, indicating the spacing between resource elements within a PRS. Allowed values: {2, 4, 6, 12}; default: 4.
- **REOffset**: Resource Element offset within the PRS comb. Range: 0 to CombSize–1; default: 0.
- **NPRSID**: PRS sequence identity, used to generate different PRS sequences. Range: 0–4095; default: 4.
- **NumPRSSymbols**: Number of OFDM symbols allocated for each PRS resource. Range: 0–12; default: 4.
- **SymbolStart**: Starting OFDM symbol index for PRS within a slot. Range: 0–13; default: 1.

3.3 sensingConfig.txt

Sensing configuration controls range/Doppler/angle FFT sizes and detection settings.

azFftLen Azimuth FFT length for beamspace processing.

elFftLen Elevation FFT length for beamspace processing.

doaEstimationMethod Angle estimation method (e.g., `beamspaceFFT` or `barlettScan`).

cfarGrdCellRange Guard cells in range bins.

cfarGrdCellVelocity Guard cells in Doppler bins.

cfarTrnCellRange Training cells in range bins.

cfarTrnCellVelocity Training cells in Doppler bins.

cfarThreshold CFAR scaling/threshold (linear or dB depending on implementation; see code).

dopplerFftLen — Doppler FFT length. Range: 1–2048; default: 64.

window — Doppler window type. Choices: {'rect', 'hamming', 'blackmanharris', 'gaussian'}; default: 'blackmanharris'.

3.4 Target Configuration (targetConfig.txt)

This file defines the ST's position and velocity as 6D coordinates as $[x, y, z, vx, vy, vz]$ where x, y, z are in m and vx, vy, vz are in m/s. There are no strict limits, but values should correspond to the simulation environment.

3.5 Base Station Configuration (bsConfig.txt)

This file defines the base station's position as 3D coordinates in meters, specified as $[x, y, z]$. There are no strict limits, but values should correspond to the simulation environment.

3.6 Channel File Specification (.json)

This section describes the JSON schema used by 5GNrad to load wideband multipath channels (e.g., 3GPP 38.901 UMa NLOS). Channel files are stored in a scenario's `/Input/` folder and are loaded automatically by `loadBackgroundChannel` and `loadTargetChannel` when you call `configScenario`.

3.6.1 What this file contains

A channel file encodes a set of multipath “drops” (or clusters), each comprising parallel arrays for delays, powers, angles, and per-ray phases. Keys observed in the provided file include: `PathDelays`, `AveragePathGains`, `AnglesAoD`, `AnglesAoA`, `AnglesZoD`, `AnglesZoA`, `InitialPhases`, `CPM`, and `HasLOSCluster`.

3.6.2 File structure

- **Container:** a JSON *array*. Each element corresponds to one multipath drop (or cluster list) at a given TX–RX pairing or time instant.
- **drop object:** each element is a JSON *object* with the fields listed below. Arrays within a drop must have the *same length* (number of rays).

3.6.3 Fields (per drop)

PathDelays Array of real numbers (seconds). Delays of each ray.

AveragePathGains Array of non-negative real numbers (linear power). One value per ray, aligned with `PathDelays`.

AnglesAoD Array of azimuth angles of departure in degrees (-180 to $+180$). One value per ray.

AnglesAoA Array of azimuth angles of arrival in degrees (-180 to $+180$). One value per ray.

AnglesZoD Array of zenith angles of departure in degrees (3GPP convention; negative offsets from boresight). One value per ray.

AnglesZoA Array of zenith angles of arrival in degrees. One value per ray.

InitialPhases Array of per-ray phases in radians (range $-\pi$ to π).

CPM Optional array holding 2×2 complex cross-polarization matrices per ray. If unused, keep as `[]`.

HasLOScluster Boolean flag indicating whether a LOS cluster is explicitly included in this drop.

3.6.4 Generating your own channels

1. **Decide the drop granularity:** one file can carry one or many drops (array length S). Use multiple drops for time evolution.
2. **Assemble per-ray arrays:** for each drop, compute N_{rays} values for delay, power (linear), azimuth/zenith AoD/AoA, and phase. Keep array lengths equal and rays aligned by index.
3. **Polarization:** if you model dual-pol coupling, encode a 2×2 CPM per ray. Otherwise leave CPM empty `[]`.
4. **LOS handling:** set `HasLOScluster=true` and include a dedicated LOS ray if your scenario requires it (UMa NLOS typically uses `false`).
5. **Place the file:** save under `<Input>/` as `targetChannel.json` or `backgroundChannel.json` (5GNrad picks it up when `configScenario` calls the background/target channel loaders).

3.7 Antenna and beamforming models (`txAntennaModel.json`, `rxAntennaModel.json`)

Each scenario provides antenna and beamforming definitions using JSON files placed in the scenario `Input/` folder: `txAntennaModel.json` for the transmitter and `rxAntennaModel.json` for the receiver. These files define the array geometry (e.g., URA dimensions and element spacing), element pattern settings, and the beamforming/combining strategy (analog, hybrid, or full digital).

3.7.1 Role in the simulation

The antenna model files are used to:

- construct MATLAB array objects (e.g., `phased.URA`, `phased.ReplicatedSubarray`, or equivalent),
- define transmit precoding vectors/matrices and receive combining vectors/matrices,
- set the number of digital output channels (N_{RF}) produced by the receiver processing chain,

3.7.2 Fields

Meta fields

<code>meta.name</code>	Identifier string (e.g., <code>"full_digital"</code> , <code>"hybrid"</code> , <code>"analog"</code>).
<code>meta.fc_Hz</code>	Design carrier frequency in Hz (used for steering vector generation).
<code>meta.array</code>	Array description: • <code>type</code>: currently <code>"URA"</code>. • <code>M</code>, <code>N</code>: vertical and horizontal element counts. • <code>Mprime</code>, <code>Nprime</code>: subarray grid size for replicated-subarray/hybrid configurations. • <code>dV_lambda</code>, <code>dH_lambda</code>: element spacing in wavelengths.

Beamformer fields

<code>beamformer.type</code>	Beamforming architecture: <code>"full_digital"</code> , <code>"hybrid"</code> , or <code>"analog"</code> .
<code>beamformer.wElem</code>	Complex element-domain weights. Representation depends on <code>beamformer.type</code> : • Analog / Full-digital: a complex vector of size <code>[Nelem x 1]</code>. • Hybrid: a complex matrix that defines the per-subarray element weights used by the replicated-subarray model (size typically <code>[NelemPerSub x Nsubarrays]</code>).

3.7.2.1 Practical notes

- Angle estimation via beamspace FFT assumes a regular grid (e.g., URA) and is most accurate for $\lambda/2$ spacing. For larger spacings, the platform may require alias handling or a scanning method (e.g., Bartlett) depending on configuration.

3.8 Runtime options (`nrRadar.run`)

The top-level runner supports name-value arguments consistent with MATLAB conventions:

"Parallel" Enable or disable parallel execution across drops. Supported values: "auto" (default), "on", "off".

Note: The platform validates configuration parameters at load time. Parameters not recognized by the current version may be ignored or may trigger a warning depending on the loader implementation.

4 Channel Inputs and Operating Modes

5G NRad supports two channel operating modes:

1. **MPC-upload mode:** If `backgroundChannel.json` and/or `targetChannel.json` are provided, 5G NRad reads per-snapshot multipath component (MPC) descriptions from these files. This mode is intended for channels generated offline (e.g., using a Release 19 TR 38.901-based sensing-capable pipeline) where large-scale effects (path loss, shadowing, LoS/NLoS state, clutter synthesis) are already embedded in the MPC gains.
2. **Simplified point-target mode:** If `targetChannel.json` is missing or empty, 5G NRad falls back to a single-specular-return target model. In this mode, the simulator computes path loss, shadow fading, and LoS probability using 3GPP models and applies an RCS-based scaling for the target return.

5 Detection and Estimation Algorithms

The core detector is implemented in `nrRadar.sens.rdmDetection`. At a high level:

1. **Range–Doppler power:** compute $P[r, d] = |X[r, d]|^2$ from the complex range–Doppler map/cube.
2. **2-D CA-CFAR:** estimate local noise power and threshold each cell under test using training and guard windows defined by `cfarTrnCell*` and `cfarGrdCell*`.
3. **Non-maximum suppression:** reduce contiguous CFAR blobs into sparse peak candidates using `nmsRadius` and `nmsMaxPeaks`.
4. **Angle estimation:** for each peak, estimate azimuth/elevation using either beamspace FFT (`doaEstimationMethod='FFT'`) or Bartlett scan (`'barlett'`).
5. **4-D clustering:** group nearby peaks in $(r, d, \text{el}, \text{az})$ bin space (DBSCAN) to produce object-level detections and suppress sidelobes/duplicates.
6. **Geometry reconstruction:** convert $(\hat{r}, \hat{\theta}, \hat{\phi})$ into Cartesian coordinates and estimate the radial velocity component.

6 Outputs

After a run, `nrRadar.io.exportResults` creates an `Output/` folder under the scenario directory and writes:

6.1 Detection payload (`Output/detection.json`)

A JSON array with one entry per drop. Each entry contains:

<code>q</code>	Drop index (1-based).
<code>targetIdx</code>	Global target indices for this drop (length <code>nStDrop</code>).
<code>gtPosition</code>	Ground-truth positions, [<code>nStDrop</code> x 3] (m).
<code>gtVel</code>	Ground-truth radial velocity per target, [<code>nStDrop</code> x 1] (m/s).
<code>stPositionEstimate</code>	Estimated detection positions, [<code>Ndet</code> x 3] (m).
<code>velParallel</code>	Estimated radial velocities for detections, [<code>Ndet</code> x 1] (m/s).
<code>peakRda</code>	Detection strength metric per detection.

6.2 Per-drop detection statistics (`Output/detStats.csv`)

A CSV table with one row per drop, containing:

<code>truePositive</code>	Number of associated detections within gate.
<code>falseNegative</code>	Number of missed ground-truth targets.
<code>falsePositive</code>	Number of detections not assigned to any ground-truth target (note the column spelling).
<code>falseAlarmProb</code>	False alarm probability, defined as $FP / \max(N_{\text{det}}, 1)$.

6.3 Per-target error table (`Output/error.csv`)

A CSV table with one row per ground-truth target per drop (total rows: `nStDrop` × `Ndrop`). If a target is not associated to any detection, its error entries are NaN.

<code>timeIndex</code>	Drop index.
<code>targetIdx</code>	Global target index.
<code>positionErrorX,Y,Z</code>	Cartesian error components (m), computed as <code>est - gt</code> .

<code>positionErrorH</code>	Horizontal position error magnitude $\sqrt{\Delta x^2 + \Delta y^2}$ (m).
<code>positionErrorV</code>	Vertical error Δz (m).
<code>rangeError</code>	Range error (m), computed relative to the TRP position.
<code>velocityError</code>	Radial velocity error (m/s), <code>est</code> - <code>gt</code> .
<code>azimuthError</code>	Azimuth angle error (deg), wrapped to $[-180, 180]$.
<code>elevationError</code>	Elevation angle error (deg), wrapped to $[-180, 180]$.
<code>TP, FN, FP, FPR</code>	Drop-level detection counts/rate replicated for convenience.
<code>snrSlice</code>	Per-target SNR value for this drop (dB).

7 Tutorial: Running the Examples

7.1 Run the full example batch

From the repository root:

```
setup();  
main
```

7.2 Run a single scenario

```
scenarioPath = "examples/uma_trp1_3gpp";  
  
[simConfig, stConfig, prsConfig, geometry, sensConfig, ...  
 backgroundChannel, targetChannel] = nrRadar.cfg.configureScenario(scenarioPath);  
  
desiredWorkers = 8;           % optional  
parallelMode    = "off";      % "on" | "off"  
  
[results, detStats, detectionOutput] = nrRadar.run( ...  
    simConfig, stConfig, prsConfig, geometry, sensConfig, ...  
    backgroundChannel, targetChannel, desiredWorkers, ...  
    "Parallel", parallelMode);  
  
nrRadar.io.exportResults(scenarioPath, results, detStats, detectionOutput);
```